

Theory

Name :- Ghulam Habib

Roll No :- 24-NTU-CS-1154

Question No :- 1

Queue :-

Queue is a linear data structure that stores element in FIFO order.

Difference :-

Stack stores element in LIFO order while queue store element FIFO order.

In queue first element inserted first and removed first.

while in a stack, the last element is inserted is removed first.

Question No :- 2 :-

FIFO (First In, First out) The first element added to queue served first.

A line at a bank teller.

The first customer join the line is served to be first.

Name: Ghulam Habib

Roll No: 24-NTU-CS-FL-1154

Question No: 3

- **Enqueue:-**

Add element at the rear of the queue.

- **Dequeue:-**

Remove an element from the front of queue.

- **Front/Ppeek:-**

Access the front element without removing it.

- **isEmpty:-**

check if Queue has no element

- **isFull:-**

check if queue has reached its maximum capacity

Question no: 4

Linear Queue

- Elements are in straight line

overflow occurs if space available at front

Circular Queue

last element is connected to first, forming a circle.

Efficient space use, reduce overflow

Name: Ghulam Habib
Roll No: 24-NTU-CS-FL-1154

implemented using a simple array.	implement with array with modulo arithmetic.
-----------------------------------	--

Question No: 5

• Advantage:-

- i) Simple to implement
- ii) Direct access to any index.

• Disadvantage:-

- Fixed size array (waste of memory or overflow)
- Inefficient in linear queues because unused front space after deque.

Question No: 6

- Each node stores data and a pointer to the next node.

• Front Pointer:- Points to first element

• Rear Pointer:- Points to the last element

• Enqueue:- add node at Rear.

• Dequeue:- Remove node from front

Advantage:- Dynamic size, no overflow unless memory is exhausted.

Name :- Ghulam Habib
Roll NO :- 24-NTU-CS-FL-1154

Question 7:-

• Priority Queue:-

Elements are removed based on priority queue not order arrival.

Difference:-

Normal Queue $\rightarrow$ FIFO order.

Priority queue:- Highest Priority served first, regardless of insertion time.

Question No: 8

Deque:-

A type of queue where insertion and deletion both operation can be done on both sides.

Operations:-

insertFront() :- Insert element at front.

insertRear() :- Insert element at rear.

deleteFront() :- Remove element from front.

deleteRear() :- Remove element from rear.

Question No: 9

circular

- Insertion at rear only
- Deletion at front
- use for fix sequence operation.

Deque

- Insertion at front and rear both
- Deletion from both rear and front
- Flexible double ended operations.

Name:- Ghulam Habib
Roll No:- 24-NTU-CS-FL-1154

Question No: 10:-

Process Scheduling:-

Ready queue stores
Processes waiting for CPU in order.
CPU picks next process from front.

Buffering:-

Queue temporarily hold
data before processing to
manage flow.

---: Scenario: ---

Question 21

• Enqueue:- A Print job enters the
rear of queue.

• Dequeue

A Printer remove job
from the front of queue.

Algorithm:-

```
while Printer is ready.  
if queue is not empty  
    job = dequeue(queue)  
    Print(job)
```

else:-

wait

Name: Ghulam Habib
Roll No: 24-NTU-CS-FL-1184

Question No: 22

- CPU uses ready queue to hold processes.
- Circular queue allows CPU to loop through processes without shifting elements.
- It gives benefit to efficient memory use. fair round robin scheduling.

Question No: 23

- Model:-
- each Patient is represented as a node in queue.
- Enqueue:-
Patient arrive and joins the rear.
- Dequeue:-
Patient at front get attended.
ensure FIFO service for fairness.

Question No: 24

Buffering:- Incoming Packet are stored in queue before being processed by router.

congestion control:- queue manage overflow packet may fall if queue get full.

Name : Ghulam Habib
Roll No: 24-NTU-CS-FL-1154

Question No 25:-

- incoming calls enter a queue in arrival order
- Processing:- Handled by agent in FIFO order.

Tracking:-

For each call, record.

waiting time:- time when service start - arrival time.

Completion time:- waiting time + service duration

Algorithm:-

initialize curtime = 0

while queue is not empty:-

call = dequeue(queue)

if call.arrivaltime > curtime

currenttime = call.arrivaltime

waiting time = currenttime - call.arrivaltime

completiontime = currenttime + call.service.

Process call.

currenttime += call.servicetime.